

CS3213: Foundations of Software Engineering

Debugging Intro and Overview



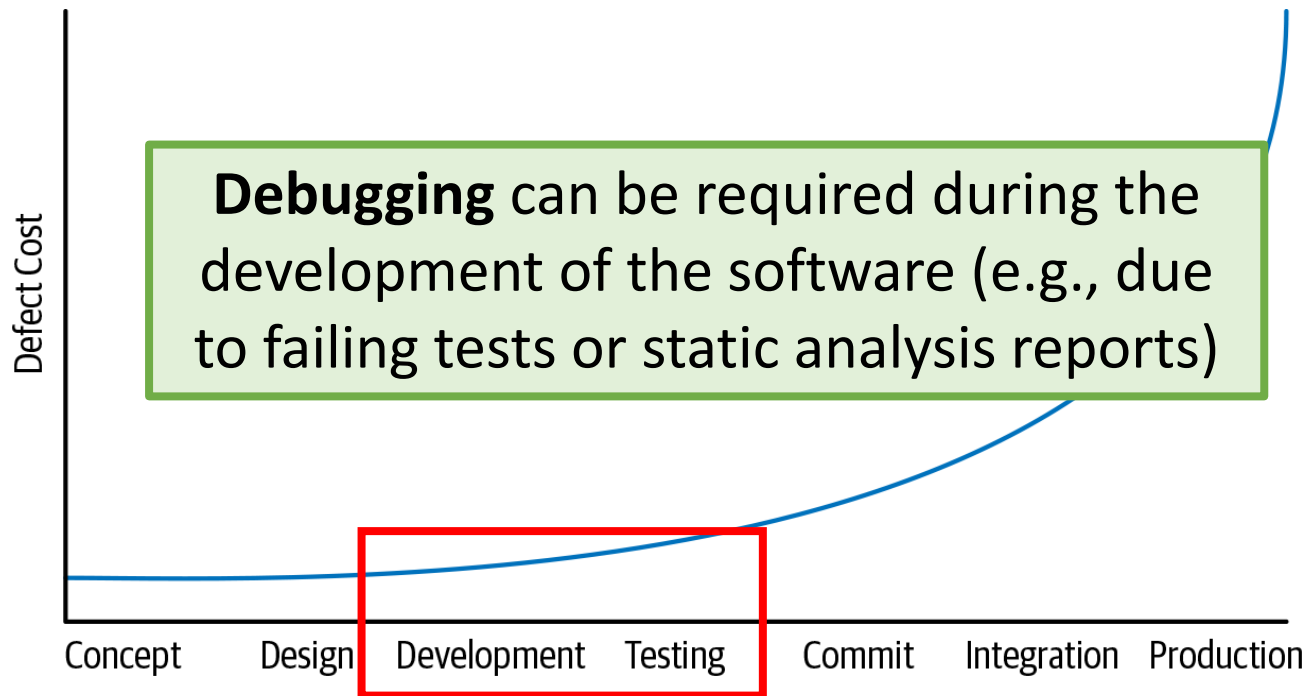
(Image by Gemini)



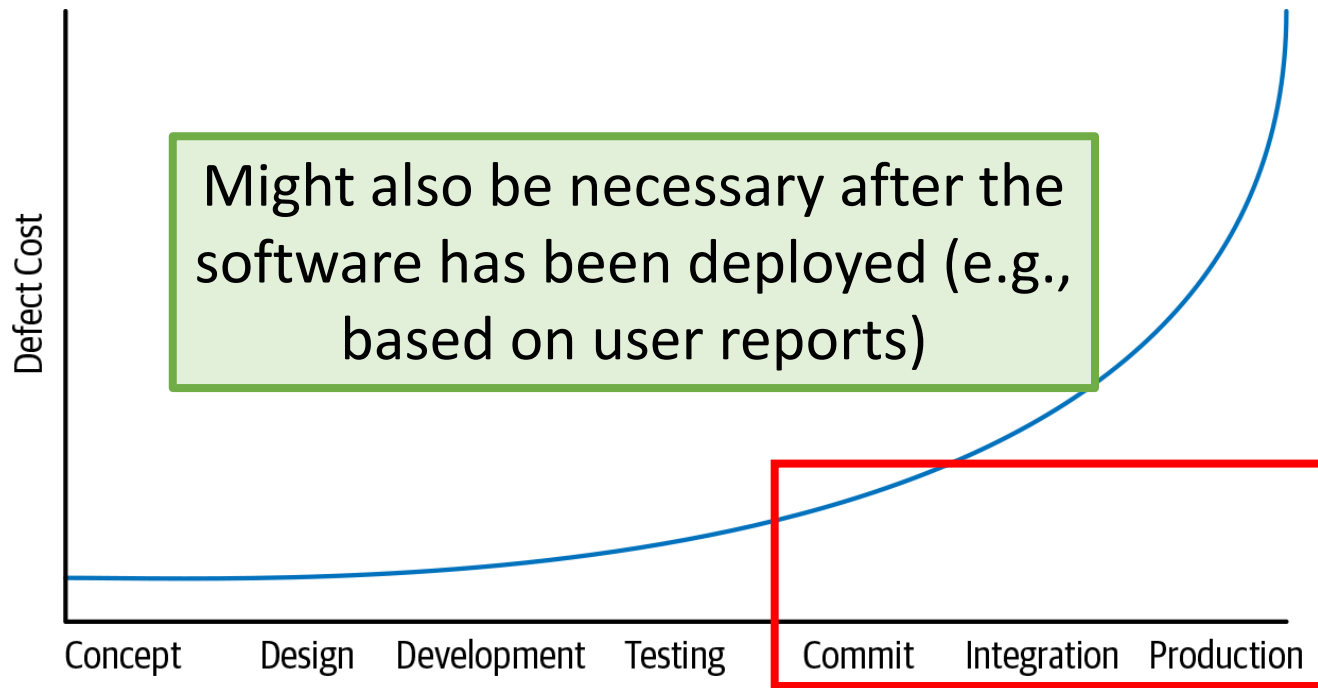
Debugging: Why?

- Software developers spend **35–50% of their time validating and debugging software**
- Estimated **cost of debugging, testing, and verification is estimated to account for 50–75%** of the total budget of software development projects
- Conclusion: identifying systematic and effective approaches to debugging greatly enhances software engineering productivity

Debugging: When?



Debugging: When?





This Week's Segments

- **Main goal:** build a mental model on how to approach debugging from different angles (i.e., *debugging strategies*)
- **Secondary goal:** some of these techniques are highly useful as tools (e.g., reduction tools)
- Understanding of important algorithms
 - Tarantula fault localization algorithm
 - Delta debugging

Example: removeHtmlMarkup

```
public static String removeHtmlMarkup(String html) {
```



```
"Here's some <strong>strong argument</strong>."
```

Assumption: we develop a method that strips away HTML tags from a string

```
return ...;
```



```
"Here's some strong argument."
```

```
}
```

Example: removeHtmlMarkup

```
public static String removeHtmlMarkup(String html) {  
    boolean tag = false;  
    String out = "";  
    for (char c : html.toCharArray()) {  
        if (c == '<') {  
            tag = true;  
        } else if (c == '>') {  
            tag = false;  
        } else if (!tag) {  
            out += c;  
        }  
    }  
    return out;  
}
```

Naïve implementation as a static method

Testing removeHtmlMarkup

Finished after 0.152 seconds

Runs: 1/1

Errors: 0

Failures: 0

TestHTMLUtils [Runner: JUnit 5] (0.026 s)

testRemoveHtmlSimple() (0.026 s)

```
@Test
void testRemoveHtmlSimple() {
    assertEquals("Here's some strong argument.",
        HTMLUtils.removeHtmlMarkup("Here's some <strong>strong argument</strong>."));
}
```

The method passes our test!

Testing removeHtmlMarkup

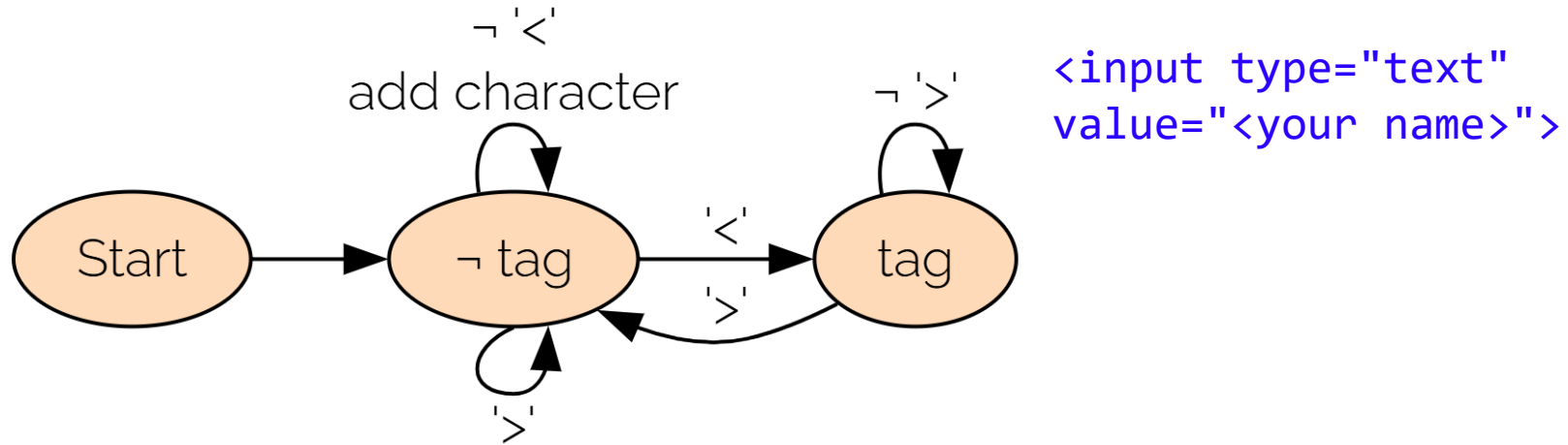
```
org.opentest4j.AssertionFailedError: expected: <> but was: <">
    at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
    at TestHTMLUtils.testRemoveHtml(TestHTMLUtils.java:14)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
```

@Test

```
void testRemoveHtml() {
    assertEquals("", HTMLUtils.removeHtmlMarkup(
        "<input type=\"text\" value=\"<your name>\">"));
}
```

A more complex test fails! How should we approach debugging?

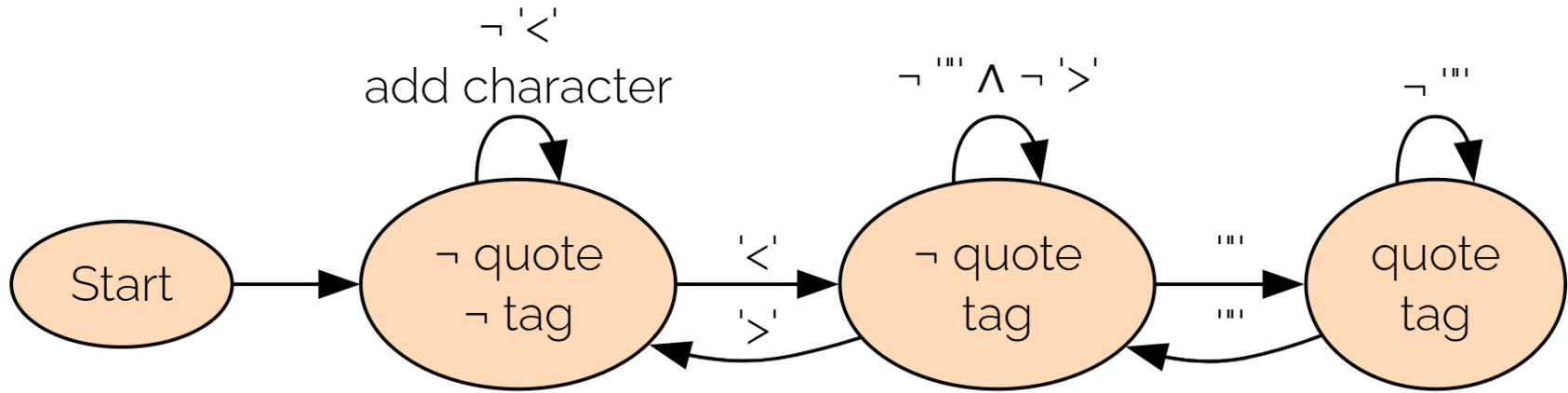
Issue in RemoveHtmlMarkup



We might realize that **for the closing tag of "<your name>", we leave the tag state** even though the tag is in quotes (i.e., not HTML markup)

Solution Idea: Quote State

`<input type="text"
value="<your name>">`



An idea to fix this issue could be **to introduce another variable** that keeps track of whether the current character/tag **is within quotes**

Fix Attempt

Finished after 0.158 seconds

Runs: 2/2

Errors: 0

Failures: 0

> TestHTMLUtils [Runner: JUnit 5] (0.034 s)

```
public static String removeHtmlMarkup(String html) {
    boolean tag = false;
    boolean quote = false;
    String out = "";
    for (char c : html.toCharArray()) {
        if (c == '<' && !quote) {
            tag = true;
        } else if (c == '>' && !quote) {
            tag = false;
        } else if (c == '"' || c == '\\' && tag) {
            quote = !quote;
        } else if (!tag) {
            out += c;
        }
    }
    return out;
}
```

Next Test Case

Finished after 0.159 seconds

Runs: 3/3

Errors: 0

Failures: 1

TestHTMLUtils [Runner: JUnit 5] (0.030 s)

testRemoveHtmlSimple() (0.016 s)

testRemoveHtmlFoo() (0.012 s)

testRemoveHtml() (0.001 s)

@Test

```
void testRemoveHtmlFoo() {  
    assertEquals("\"foo\"", HTMLUtils.removeHtmlMarkup("<b>\"foo\"</b>"));  
}
```

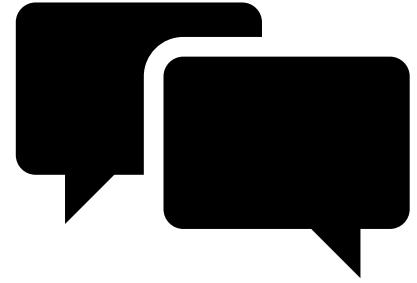


How Not To Approach Debugging

- We can change the code and try various conditions until it works
- Issue: if we do not understand the underlying cause, we might not fix the bug's underlying root cause



Reflection



How to approach debugging in a more systematic way?

Printf Debugging

```
public static String removeHtmlMarkup(String html) {  
    boolean tag = false;  
    boolean quote = false;  
    String out = "";  
    for (char c : html.toCharArray()) {  
        System.out.printf("c=%c tag=%b quote=%b\n", c, tag, quote);  
        if (c == '<' && !quote) {  
            tag = true;  
        } else if (c == '>') {  
            tag = false;  
        } else if (c == '\"') {  
            quote = !quote;  
        } else if (!tag) {  
            out += c;  
        }  
    }  
    return out;  
}
```

```
c=< tag=false quote=false  
c=b tag=true quote=false  
c=> tag=true quote=false  
c=" tag=false quote=false  
c=f tag=false quote=true  
c=o tag=false quote=true  
c=o tag=false quote=true  
c=" tag=false quote=true  
c=< tag=false quote=false  
c=/ tag=true quote=false  
c=b tag=true quote=false  
c=> tag=true quote=false
```

Printf Debugging

printf

<stdio.h>

```
int printf ( const char * format, ... );
```

Print formatted data to stdout

Writes the C string pointed by **format** to the standard output ([stdout](#)). If **format** includes **format specifiers** (subsequences beginning with %), the additional arguments following **format** are formatted and inserted in the resulting string replacing their respective specifiers.

The name “printf” debugging stems from the printf function in C



Printf Debugging

- **Pros**

- Simple and intuitive
- Language and tool agnostic

- **Cons**

- Can quickly become confusing and unclear
- Might forget print statements in the program code
- Might require recompilation of the code



Logging

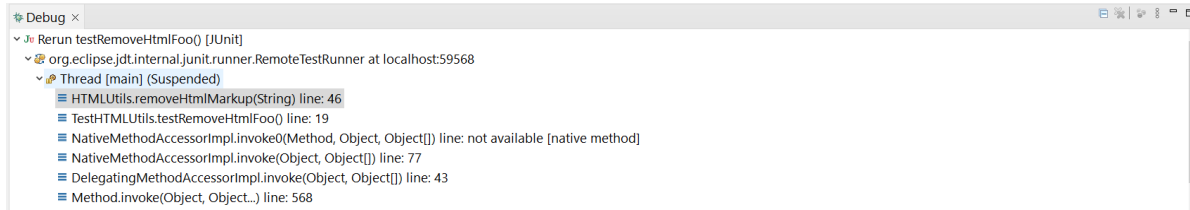
- More systematic alternative to adding print statements
- Typically, multiple debugging levels are supported

Message type	Description
DEBUG	A general debugging event.
INFO	An event for informational purposes.
WARN	An event that might possible lead to an error.
ERROR	An error in the application, possibly recoverable.
FATAL	A fatal event that will prevent the application from continuing.

Debugger

```
public static String removeHtmlMarkup(String html) {  
    boolean tag = false;  
    boolean quote = false;  
    String out = "";  
    for (char c : html.toCharArray()) {  
        if (c == '<') {  
            tag = true;  
        } else if (c == '>') {  
            tag = false;  
        } else if (c == '"') {  
            quote = !quote;  
        } else if (!tag) {  
            out += c;  
        }  
    }  
    return out;  
}
```

Using a language-specific debugger





Debugging: Functionality

- Setting ***breakpoints***: pausing at certain locations in the code
- ***Stepping*** in/over/out of methods
- Step-by-step execution
- Inspecting program state (e.g., variable values, stack trace)
- ...



Debugger

- **Pros**

- More systematic than printf debugging
- Can display additional information (e.g., stack trace, values of variables and intermediate results)
- Can set breakpoints that are specific to values



Common Tools and Approaches

- Printf debugging
- Logging
- Language-specific debuggers

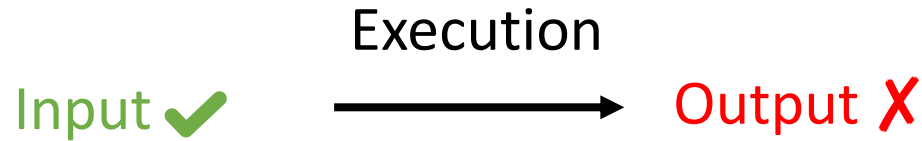
We still require a debugging strategy!



Terminology

- **Mistake:** act or decision resulting in an error
- **Defect (bug):** an error in the program code
- **Fault:** the location in which the programming error is triggered and we enter an unwanted state
- **Failure:** where the fault becomes externally visible

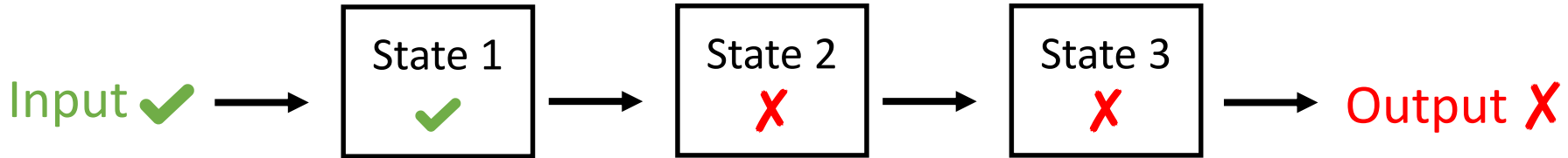
Terminology and Mental Model



Through the test, we observed a *failure*.
Now, we want to find the *defect*.

Terminology and Mental Model

Execution



Mistake → Defect → Fault → ... → Fault → Failure

In order to debug the failure, we need to trace it back to the **defect** that caused the first fault

Challenges

- **Program states** are typically **large**, making it challenging or infeasible to search all of them manually
- It is **not always clear** whether (intermediate) states are **correct or not**
- Executions might consist of **million of steps**

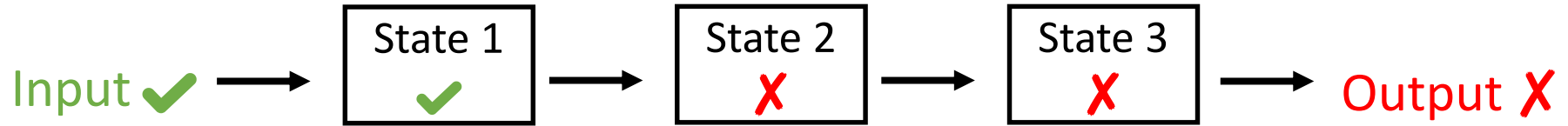
Given these challenges, we require a **debugging strategy** that helps us focus the search!



“The Scientific Method” to Debugging

1. Formulate a question
2. Come up with an ***hypothesis*** based on the knowledge obtained while formulating the question that may explain the observed behavior
3. Determining the logical consequences of the hypothesis, **formulate a prediction that can support or refute the hypothesis**. Ideally, the prediction would distinguish the hypothesis from likely alternatives.
4. **Test the prediction** (and thus the hypothesis) in an experiment. If the prediction holds, confidence in the hypothesis increases; otherwise, it decreases.
5. Repeat Steps 2–4 until there are no discrepancies between hypothesis and predictions and/or observations.

Tracking Origins Debugging Strategy



- Debugging strategy: “tracking origins”
- We start with a single faulty state f – the failure.
- We determine f 's **origins** – the parts of earlier states that could have caused the faulty state f .
- For each of these origins e , we determine whether they are faulty or not, until we have found the defect.

Program Slicing

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

```
int m = middle(2, 1, 3);  
System.out.println(m);
```

A *slice* identifies the subset of the program that could have influenced the variable/returned value

Statistical Fault Localization: Tarantula

Line	abc ✓	abc ✓	"abc" ✗
1 (boolean tag = false;)	X	X	X
2	X	X	X
3	X	X	X
4	X	X	X
$\text{Suspiciousness}(10) = \frac{\frac{\text{failed}(10)}{\text{total failed}}}{\frac{\text{failed}(10)}{\text{total failed}} + \frac{\text{passed}(10)}{\text{total passed}}} = \frac{\frac{1}{1}}{\frac{1}{1} + \frac{0}{2}} = 1$			X
8 (tag = false;)	-	X	-
9	X	X	X
10 (quote = !quote;)	-	-	X
11	X	X	X
12	>		
15	>		

Highly suspicious line, as it was covered only in failing tests

$$c_{\mathbf{x}}' = \langle \text{SELECT} \rangle$$

Delta Debugging: Test-case Reduction

- $\Delta_1 = <, \Delta_2 = S, \Delta_3 = E, \Delta_4 = L, \Delta_5 = E, \Delta_6 = C, \Delta_7 = T, \Delta_8 = >$
- $\nabla_1 = \text{SELECT}, \nabla_2 = \langle \text{ELECT} \rangle, \nabla_3 = \langle \text{SLECT} \rangle, \nabla_4 = \langle \text{SEECT} \rangle, \nabla_5 = \langle \text{SELCT} \rangle, \nabla_6 = \langle \text{SELET} \rangle, \nabla_7 = \langle \text{SELEC} \rangle, \nabla_8 = \langle \text{SELECT} \rangle$

No other steps are applicable, so $\langle \text{SELECT} \rangle$ is the 1-minimal bug-inducing test case



Minimizing Delta

Let test and $c_{\mathbf{x}}$ be

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \mathbf{x}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$

$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \mathbf{x} \wedge n \leq |c_{\mathbf{x}}'|$.

Main Source

The Debugging Book

Tools and Techniques for Automated Software Debugging

by Andreas Zeller

About this Book

Welcome to "The Debugging Book"! Software has bugs, and finding bugs can involve lots of effort. This book addresses this problem by automating software debugging, specifically by *locating errors and their causes automatically*. Recent years have seen the development of novel techniques that lead to dramatic improvements in automated software debugging. They now are mature enough to be assembled in a book - even with executable code.

```
from bookutils import YouTubeVideo
YouTubeVideo("-m0x16Ev_14")
```



The Debugging Book



Watch on 

debuggingbook.org
Music: bensound.com

A Textbook for Paper, Screen, and Keyboard

You can use this book in four ways:

- You can **read chapters in your browser**. Check out the list of chapters in the menu above, or start right away with the [introduction to debugging](#) or [how debuggers work](#). All code is available for download.
- You can **interact with chapters as Jupyter Notebooks** (beta). This allows you to edit and extend the code, experimenting *live in your browser*. Simply select "Resources → Edit as Notebook" at the top of each chapter. [Try interacting with the introduction to interactive debuggers](#).
- You can **use the code in your own projects**. You can download the code as Python programs; simply select "Resources → Download Code" for one chapter or "Resources → All Code" for all chapters. These code files can be executed, yielding (hopefully) the same results as the notebooks. Once the book is out of beta, you can also [install the Python package](#).
- You can **present chapters as slides**. This allows for presenting the material in lectures. Just select "Resources → View slides" at the top of each chapter. [Try viewing the slides for how debuggers work](#).

<https://www.debuggingbook.org/>

Related Videos

The image shows a YouTube video player interface. The video title is "Statistical Debugging". A vertical green timeline on the left side of the video frame marks the years "1960" at the top and "2001" at the bottom, with a green arrow pointing downwards. The video content includes a bulleted list:

- **Intuition:** debugging techniques can leverage multiple executions
- Tarantula

At the bottom of the video frame, a red progress bar is visible, with a playhead icon and the text "20:58 / 1:05:40 • Statistical Debugging >". To the right of the video frame, there is a vertical stack of four video call windows. The top window shows a cartoon character, the second shows a man with glasses, the third shows a man with glasses, and the bottom window shows a man with glasses. The bottom of the video player features standard YouTube controls: play/pause, next, volume, full screen, and other icons.



Summary and Takeaway

- We spent much time debugging
- Issue with ad-hoc debugging
- Printf debugging, logging, and debuggers
- Debugging strategies and automated debugging techniques can help with debugging